

SEARCHING

PROBLEM 10

QUESTION:

DS	Session	Searching	Question Information	Level 1	Challenge 10
Question Description: An array A contains integers with the following constraints: 'A' contains elements in sorted order. Integer i occurs $i \cdot \text{floor}(\sqrt{i}) + \text{ceil}(i/2)$ times in the array. All elements are greater than or equal to 1. Fathima has given Q queries of type: L R: Find the number of distinct values present in subarray $A[L..R]$. Note: 1-based indexing is followed. Constraints: $1 \leq Q \leq 10^5$ $1 \leq L \leq R \leq 10^{13}$ Input Format: The first line contains an integer Q denoting the number of queries. Next Q lines contains two space-separated integers L, R, denoting the query. Output Format: Print the output in separate lines contains required number of distinct values. Example Sample Input: <pre>2 1 3 1 6</pre> Sample Output: <pre>2</pre>					

3
Explanation First few elements of the array A are 1, 1, 2, 2, 2, 3, 3, 3, 3, 3, ... - For Query 1:- - Number of distinct elements in subarray $A[1..3]$ is 2. - For Query 2:- - Number of distinct elements in subarray $A[1..6]$ is 3.
Logical Test Cases Test Case 1 INPUT (STDIN) <pre>2 3 3 2 6</pre> EXPECTED OUTPUT <pre>1 3</pre> Test Case 2 INPUT (STDIN) <pre>3 3 3 2 6 5 4</pre> EXPECTED OUTPUT <pre>1 3 1</pre>
Mandatory Test Cases Test Case 1 KEYWORD <pre>while(l<ans1)</pre>
Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY <pre>6</pre> Test Case 2 TOKEN COUNT <pre>400</pre> Test Case 3 NLOC <pre>56</pre>

SOURCE CODE:

```
#include <stdio.h>
#include <math.h>

long long compute_element_count(long long val) {
    return val * (long long)floor(sqrt(val)) + (val + 1) / 2;
}

long long locate_value_at(long long target_index) {
    long long left = 1, right = 1000000, best_val = 1;
    while (left <= right) {
        long long mid = left + (right - left) / 2;
        long long running_sum = 0;
        for (long long i = 1; i <= mid; i++) running_sum += compute_element_count(i);
        if (running_sum >= target_index) {
            best_val = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    return best_val;
}

int main() {
    int num_queries;
    if (scanf("%d", &num_queries) != 1) return 0;

    while (num_queries--) {
        long long left_pos, right_pos;
        scanf("%lld %lld", &left_pos, &right_pos);
        long long val_start = locate_value_at(left_pos);
        long long val_end = locate_value_at(right_pos);
        printf("%lld\n", val_end - val_start + 1);
    }
    return 0;
}
```

OUTPUT:

```
2
1 6
2 6
3
2
```

```
2
5 4
3 1
1
3
```